Algoritmos Distribuidos
Sincronización de Relojes

Andrés Sanoja

Claudia León

Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación

Octubre 2008 – Marzo 2009



Objetivos

- Comprender y describir el problema de sincronización de relojes lógicos en sistemas distribuidos
- Comprender y describir diferentes soluciones para sincronizar relojes lógicos.

Sincronización de relojes

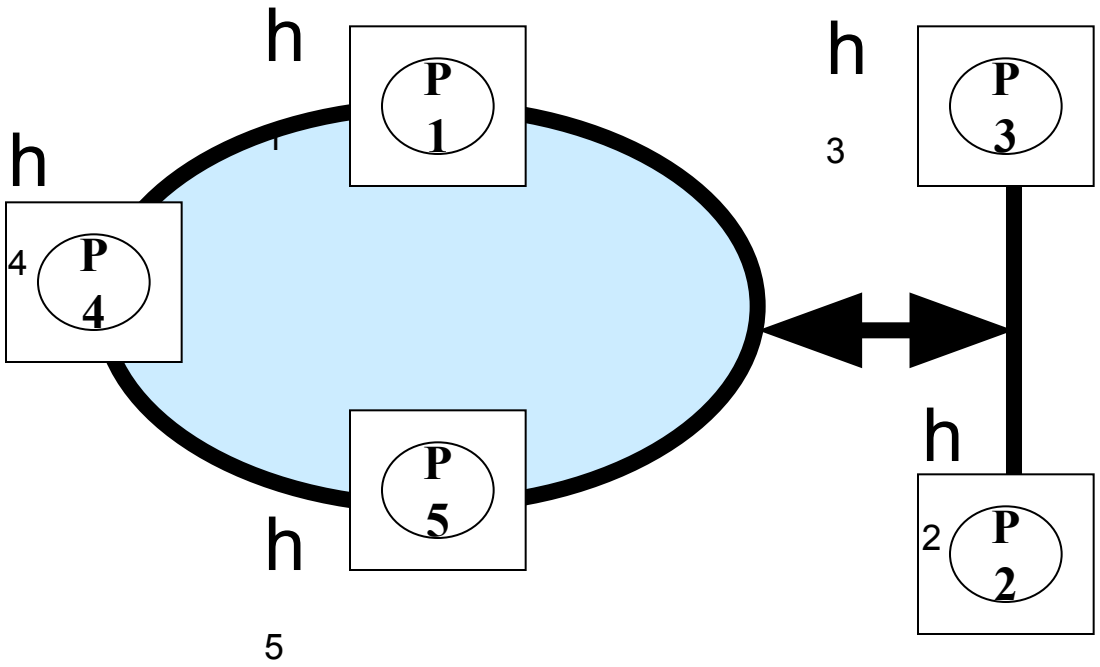
Problema: Se tienen n nodos numerados de 1 a n, cada uno equipado con un reloj lógico h_i.
Se desea establecer un control sobre el incremento de estos relojes de forma tal que se mantenga la siguiente invariante:

$$\forall\ i, j \in 1..n : |h_i - h_j| \leq \delta, \text{ para una constante } \delta \in N$$

Ejemplo de Aplicación: Si P_i,P_j son procesos que solicitan una operación sobre un recurso, se quiere que en todo momento cada uno de los procesos no pueda emitir más de δ solicitudes con respecto a otro.

La solución debe garantizar la mayor autonomía posible entre los procesos.

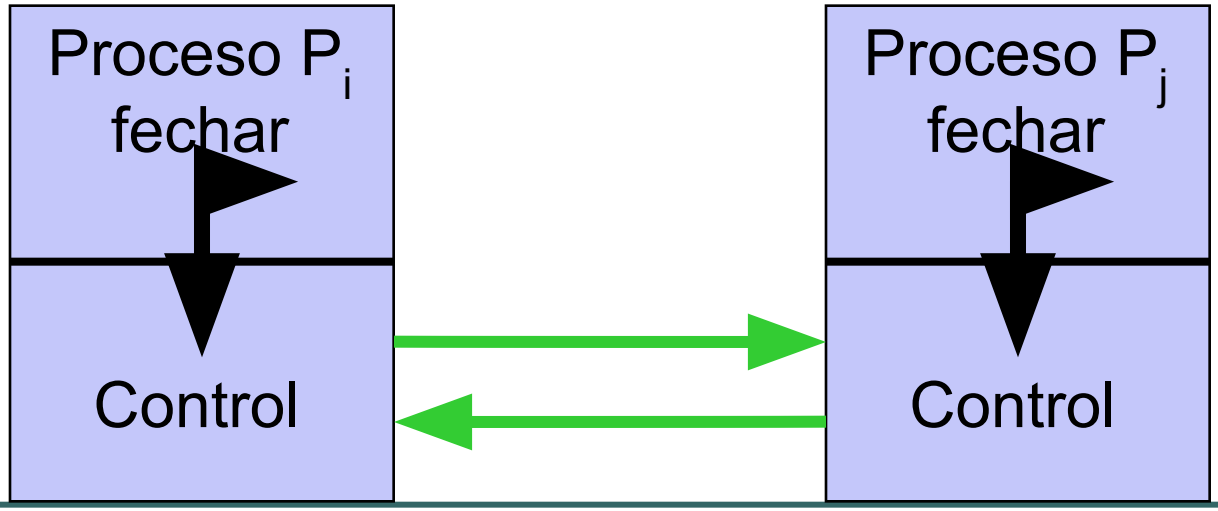
Sincronización de relojes



Podemos separar el valor del reloj de la preservación de la invariante, introduciendo en cada proceso P_i una variable local max_i que indique en todo momento el valor límite hasta el cual puede incrementarse el reloj local h_i sin que se invalide la invariante.

Sincronización de relojes: Solución de Carvalho-Roucairol

En cada nodo se tienen dos niveles en el algoritmo:
1.Utilización del servicio (fechar): Interfaz que permite a un proceso obtener el valor del reloj local
2. Realización del servicio (Control): Implementación del mantenimiento de la invariante sobre los relojes



Hipótesis del algoritmo

- Es posible duplicación ó pérdida de mensajes, ó pérdida de secuencia
- Se asume una malla completa como topología lógica
- La operación *fechar* tiene asociada la condición: $h_i < max_i$
- La invariante sobre un nodo cualquiera P_i es:

$$\forall\ i, j \in 1..n : |h_i - h_j| \leq \delta$$

- Es necesario que P_i capture el estado de los otros nodos. Para ello:
  - Difusión: P_i comunica los valores a P_j, $\forall\ j$
  - Memoria: P_i almacena los más grandes valores del reloj de P_j, $\forall\ j$

Algoritmo para sincronizar n relojes Carvalho-Roucairol

- En todo momento P_i puede difundir el valor de su reloj a los otros P_j.
- Ultimo[j] contiene el mayor valor del reloj de P_j recibido por P_i
- h_i está restringida por max_i de la siguiente forma:

  $h_i \leq max_i \leq$ Minimo{Ultimo[j]+ δ } i $\neq$j

  El Mínimo corresponde al nodo cuyo reloj progresa más lento o donde la comunicación del valor del reloj es más lenta.

Algoritmo para sincronizar n relojes (cont.)

```
 Var h,y : entero Inicializa 0; max: entero inicializaδ
 Ultimo arreglo [P:procesos] de enteros inicializa 0;
Cuando {se llame a fechar} hacer
   Si h < max entonces h := h+1; fsi
fcuando

Cuando {desee difundir} hacer //previo: llamar a fechar
   paratodo j ∈ vecinos hacer enviar(h) a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
     recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
    Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

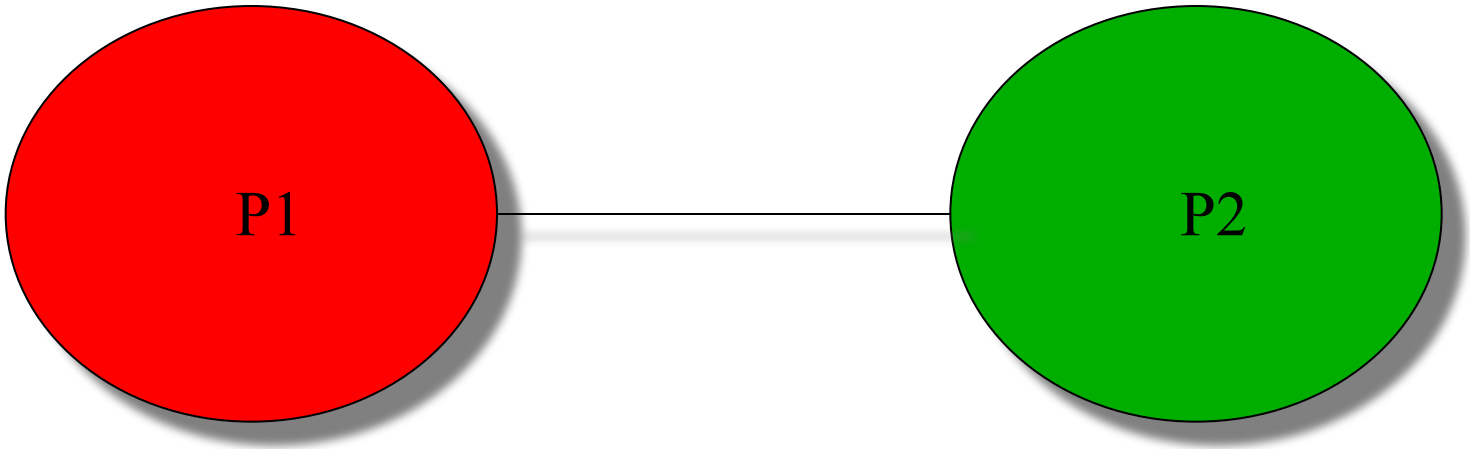
Algoritmo para sincronizar n relojes Carvalho-Roucairol

El algoritmo de Carvalho-Roucairol propone resolver este problema a través del pase periódico de mensajes entre procesos.

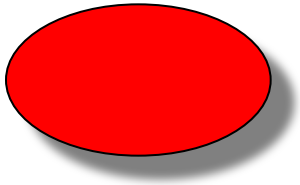
Esta solución incrementa el tráfico dentro de los canales de comunicación.



Corrida. Topología



Corrida P1



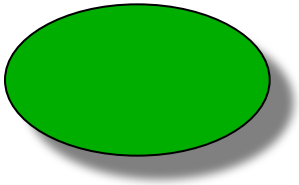
h=0
y=0
Ultimo={0}
δ=2
max=0+δ=2

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
Si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P2



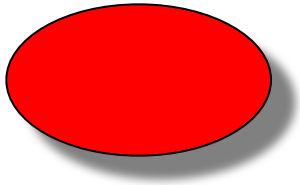
h=0
y=0
Ultimo={0}
δ=2
max=0+δ=2

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
Si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P1



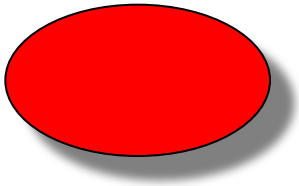
h=1
y=0
Ultimo={0}
δ=2
max=0+δ=2

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
Si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P1



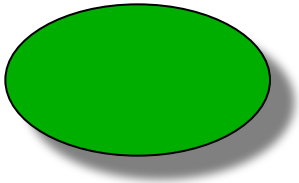
h=1
y=0
Ultimo={0}
δ=2
max=0+δ=2

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
Si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P2



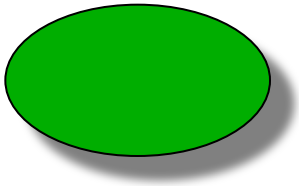
h=0
y=0
Ultimo={0}
δ=2
max=0+δ=2
x=1

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
Si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P2



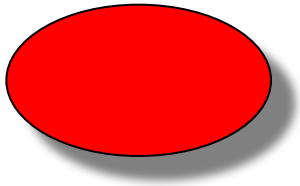
```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

h=0
y=min(1)=0
Ultimo={1}
δ=2
max=1+δ=3
emisor=1
Ultimo[1]=0
$x=h_1=1$

Corrida P1



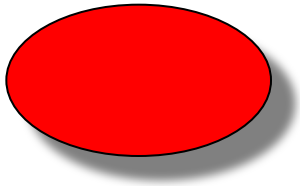
h=2
y=0
Ultimo={0}
δ=2
max=0+δ=2

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P1



h=2
y=0
Ultimo={0}
δ=2
max=0+δ=2

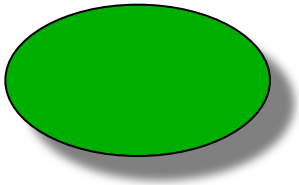
```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {l                              er
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

h>=max

Corrida P2



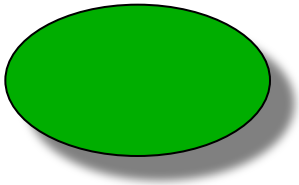
```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

h=1
y=min(1)=0
Ultimo={1}
δ=2
max=1+δ=3
emisor=1
Ultimo[1]=0
$x=h_1=1$

Corrida P2



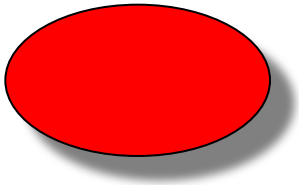
```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

h=1
y=min(1)=0
Ultimo={1}
δ=2
max=1+δ=3
emisor=1
Ultimo[1]=0
$x=h_1=1$

Corrida P1



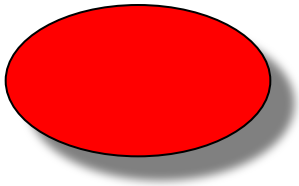
h=2
y=0
Ultimo={0}
δ=2
max=0+δ=2

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P1



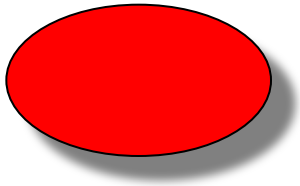
h=2
y=Min(2)
Ultimo={2}
δ=2
max=2+δ=4

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P1



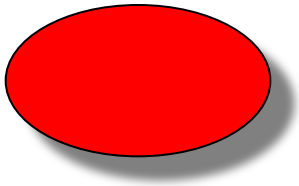
h=2
y=Min(2)
Ultimo={2}
δ=2
max=2+δ=4

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {llegue un mensaje} hacer
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

Corrida P1



h=3
y=Min(2)
Ultimo={2}
δ=2
max=2+δ=4

```
 Var h,y : entero Inicializa 0; max: entero
 Ultimo arreglo [P:procesos] de enteros inic
Cuando {se llame a fechar} hacer
si h < max entonces h := h+1;  retorna(h) fsi
fcuando

Cuando {desee difundir} hacer
paratodo j ∈ vecinos hacer enviar(h)a j; fparatodo;
fcuando

Cuando {l                                  er
  recibir (x) de emisor;
     si  Ultimo[emisor] < x entonces
Ultimo[emisor]:=x;  y:=Mínimo{Ultimo[j],∀ j}; max:=y + δ
     fsi
fcuando
```

h<max